Exact count-n EFX instances for additive valuations beyond $m = n + 2$ and a bounded counting verification

Specialist review copy

27 September 2026

Abstract

Neoh and Teh initiated the systematic study of how many allocations satisfy envy-freeness up to any good (EFX). Their Conjecture 6.1 predicted at least n EFX allocations for general monotone valuations, but the 2026 monotone submodular counterexamples of Akrami et al. refuted that statement. Its additive specialization remains open. We give positive additive instances having exactly n labeled EFX allocations at $(n, m) = (4, 7), (5, 8), (6, 9)$ and at $(4, 8), (4, 9), (4, 10)$. Thus the minimum over additive instances is at most n at these six cells; equality remains conditional on the additive lower-bound conjecture. The allocations in these instances are explained by a generalized parking criterion: for an EFX allocation of all but one good, an explicit observer–host inequality is necessary and sufficient for placing the remaining good at a chosen host. Finally, a slot-protected counterexample-guided computation reports that every positive integer $(4, 7)$ instance with values in $\{1, \ldots, 4\}$ has at least four EFX allocations. We prove the soundness of the slot encoding and provide exact instance verifiers, parking-slack certificates, and a reconstruction and replay package. Because the historical run did not retain its final checkpoint, we regenerated an alternative complete terminal pool; the fresh CP-SAT master is infeasible at pool size 1,240, and an independently encoded CNF master is also UNSAT under CaDiCaL. The artifact archives CaDiCaL's binary DRAT proof, independently verified by `drat-trim` [17]. The bounded result depends on the stated CNF translation and proof checker; it is not an unrestricted counting theorem.

1 Introduction

Let $N = [n]$ be a set of agents and $M = [m]$ a set of indivisible goods. Agent i has an additive valuation $v_i \colon 2^M \to \mathbb{R}_{\geq 0}$, with $v_i(S) = \sum_{g \in S} v_i(g)$. An allocation is a labeled partition $X = (X_1, \ldots, X_n)$ of M.

Definition 1.1. An allocation X is EFX_0 if, for every pair $i \neq j$ and every $g \in X_j$,

$$v_i(X_i) \;\geq\; v_i(X_j \setminus \{g\}).$$

The subscript distinguishes the strong convention in which zero-valued goods are also quantified over. Every computational instance and bounded result in this paper uses positive values, so the distinction disappears there; the parking theorem is stated directly for EFX_0 and permits nonnegative valuations. We count labeled allocations: permuting bundles among agents generally produces a different allocation.

Suksompong [1] obtained tight counting results for two agents. Neoh and Teh [2] subsequently studied the minimum count as a function of (n, m) for general monotone valuations, using additive

examples for their upper bounds. Their Table 1 gives the exact value n at $m = n + 1$, bounds $[n, n^2]$ at $m = n+2$, and groups $m \geq n+3$ under the upper bound $n!$ with no positive lower bound uniform across that entire regime. They conjectured at least n allocations without an additivity restriction. Akrami et al. [11] have since constructed monotone submodular instances with no EFX allocation for $n \geq 3$ and $m \geq n + 5$, refuting that original conjecture.

Conjecture 1.2 (additive specialization of Neoh–Teh [2], Conjecture 6.1)**.** *With $n \geq 1$ agents, at least one good, and additive valuations, there are at least n EFX_0 allocations.*

Conjecture 1.2 is the still-open additive restriction, not the now-false general-monotone statement. It includes additive EFX existence as a special consequence. For clarity, call an additive instance *count-n* if its complete labeled allocation set contains exactly n EFX_0 allocations. We do not call such an instance tight unless a matching additive lower bound is available.

Contributions. The paper has three claims.

(i) We give explicit count-n instances at six cells:

$$(4, 7),\ (5, 8),\ (6, 9),\ (4, 8),\ (4, 9),\ (4, 10).$$

Two independently written exact verifiers and a third vectorized implementation enumerate all n^m labeled allocations and agree on every count. These are upper bounds on the additive minimum count; they do not prove Conjecture 1.2.

(ii) We prove a generalized parking theorem. It gives an exact observer–host inequality for when an unallocated good may be added to a bundle while preserving EFX_0. Machine-checkable slack matrices show that all n parkings in each displayed instance satisfy the theorem. Universal-minimum parking is only a sufficient special case, and is not assumed for the family.

(iii) We formalize a slot-protected counting-CEGAR master and prove its soundness. CP-SAT and an independently encoded threshold-CNF master both report UNSAT over the finite box $\{1, \ldots, 4\}^{4\times 7}$ for the hypothesis "at most three EFX_0 allocations." The artifact audits the historical checkpoint, supplies a complete alternative terminal pool generated by atomic reconstruction, and archives both masters, their replay programs, and an independently checked binary DRAT certificate for the CNF master.

The instance upper bounds and bounded lower bound are logically separate. In particular, the printed $(4, 7)$ instance has values well outside $\{1, \ldots, 4\}$, so the two computations cannot be combined into an unrestricted equality statement.

Related computational work. For additive valuations, EFX exists for at most three agents [4]. Mahara [3] proves EFX existence when $m \leq n+3$, so existence is unconditional in our three diagonal cells even for general monotone valuations. More recently, Alkassar, Fouz and Mehlhorn [5] prove complete EFX_0 existence for four additive agents and at most nine goods, including nonnegative real valuations. Our claims at $(4, 8)$ and $(4, 9)$ concern the number of allocations, not a new existence frontier; the bounded $(4, 7)$ result likewise certifies a count of at least four, not merely existence. Akrami et al. [11] refute general-monotone EFX existence from $m = n + 5$ onward, while explicitly leaving additive valuations outside their counterexamples. Their work also gives a recent computer-assisted precedent: a Lean-checked theoretical foundation for the SAT encoding, two SAT solvers, and a DRAT-checked UNSAT proof. Our finite result concerns bounded additive integer valuations

and now also retains an independently checked DRAT proof, but no proof-assistant formalization of its mathematical encoding. The witness search is also distinct from Brânzei's simulated-annealing search for an EFX allocation of a fixed instance [10]: here the search varied valuation matrices to minimize the exact allocation count.

2 Six count-n cells

For an additive instance I, write $\#\mathrm{EFX}_0(I)$ for its number of complete labeled EFX_0 allocations, and let the *additive minimum count* be the minimum of this quantity over additive instances with the same (n, m). An explicit instance with $\#\mathrm{EFX}_0(I) = n$ proves that this additive minimum is at most n, regardless of whether the additive lower bound is 0, lies in $\{1, \ldots, n-1\}$, or equals n.

Computer-verified result 2.1 (count-n instances)**.** There are positive additive instances with exactly n EFX_0 allocations at $(4, 7)$, $(5, 8)$, and $(6, 9)$, and positive additive instances with exactly four EFX_0 allocations at $(4, 8)$, $(4, 9)$, and $(4, 10)$.

Rows below are agents and columns are goods. One witness for each cell is:

$$(4,7): \begin{bmatrix} 30 & 66 & 69 & 9 & 21 & 19 & 63 \\ 12 & 56 & 58 & 9 & 10 & 24 & 52 \\ 12 & 33 & 45 & 20 & 24 & 8 & 64 \\ 11 & 60 & 1 & 18 & 2 & 6 & 41 \end{bmatrix}$$

$$(5,8): \begin{bmatrix} 32 & 4 & 10 & 60 & 8 & 3 & 16 & 11 \\ 42 & 3 & 57 & 50 & 51 & 1 & 5 & 10 \\ 54 & 12 & 4 & 27 & 3 & 20 & 18 & 11 \\ 33 & 9 & 31 & 42 & 45 & 10 & 13 & 5 \\ 45 & 13 & 52 & 47 & 49 & 21 & 2 & 21 \end{bmatrix}$$

$$(6,9): \begin{bmatrix} 15 & 86 & 103 & 193 & 191 & 192 & 195 & 196 & 1 \\ 6 & 3 & 12 & 105 & 23 & 65 & 4 & 2 & 1 \\ 11 & 68 & 55 & 10 & 152 & 8 & 7 & 2 & 1 \\ 49 & 3 & 11 & 2 & 64 & 104 & 8 & 7 & 1 \\ 2 & 6 & 31 & 79 & 40 & 61 & 104 & 7 & 1 \\ 53 & 45 & 3 & 105 & 102 & 103 & 151 & 152 & 2 \end{bmatrix}.$$

For the three remaining cells:

$$(4,8): \begin{bmatrix} 11 & 8 & 20 & 1 & 4 & 1 & 16 & 1 \\ 16 & 49 & 32 & 2 & 16 & 2 & 60 & 1 \\ 28 & 28 & 59 & 1 & 60 & 7 & 46 & 8 \\ 10 & 12 & 24 & 1 & 59 & 1 & 17 & 1 \end{bmatrix}$$

$$(4,9): \begin{bmatrix} 1 & 57 & 58 & 6 & 60 & 7 & 19 & 14 & 15 \\ 1 & 57 & 11 & 12 & 4 & 12 & 1 & 21 & 10 \\ 4 & 59 & 60 & 2 & 23 & 3 & 28 & 8 & 12 \\ 8 & 48 & 57 & 1 & 58 & 34 & 1 & 1 & 3 \end{bmatrix}$$

$$(4,10): \begin{bmatrix} 17 & 57 & 58 & 3 & 47 & 1 & 6 & 23 & 3 & 1 \\ 2 & 51 & 57 & 5 & 58 & 11 & 12 & 1 & 10 & 10 \\ 1 & 75 & 15 & 8 & 25 & 13 & 19 & 6 & 12 & 16 \\ 10 & 58 & 59 & 8 & 60 & 2 & 8 & 7 & 22 & 6 \end{bmatrix}.$$

Verification. Verifier 1 enumerates assignments by a base-n counter and checks bundle bitmasks. Verifier 2 uses a recursive good-by-good enumeration and separately constructs observer–bundle totals and minima. A vectorized counter supplies a third implementation. All arithmetic is integral. For each observer–target pair, the strongest EFX_0 constraint is obtained by removing an observer-minimum good from the target bundle, so the optimized check is

$$v_i(X_i) \;\geq\; v_i(X_j) - \min_{g \in X_j} v_i(g).$$

This optimized constraint is imposed only when X_j is nonempty; an empty target bundle has no removable-good constraint. The second verifier is also tested against a definition-faithful loop that checks every removable good separately. The artifact contains the instance files, both exact verifiers, expected outputs, and hashes.

The instances were found by directed count-minimizing search. Search restrictions do not enter Result 2.1: once a matrix is printed, exhaustive enumeration alone establishes its count.

(n, m)	allocations checked	$\#\mathrm{EFX}_0$	prior additive UB	artifact instance
$(4, 7)$	$4^7 = 16{,}384$	4	24	`t47.json`
$(5, 8)$	$5^8 = 390{,}625$	5	120	`ch5_A.json`
$(6, 9)$	$6^9 = 10{,}077{,}696$	6	720	`tight69_witness.json`
$(4, 8)$	$4^8 = 65{,}536$	4	24	`tb4.json`
$(4, 9)$	$4^9 = 262{,}144$	4	24	`tight49_witness.json`
$(4, 10)$	$4^{10} = 1{,}048{,}576$	4	24	`tight410_witness.json`

Table 1: Full labeled-allocation enumerations for the six cells. "Prior additive UB" is the $n!$ upper bound witnessed by the additive examples of Neoh and Teh [2].

3 A generalized parking theorem

Let f be a good and let $B = (B_1, \ldots, B_n)$ allocate $M \setminus \{f\}$. For host h, write B^{+h} for the complete allocation obtained by adding f to B_h. For observer $k \neq h$, define the *parking slack*

$$\sigma_{k,h}(B, f) = v_k(B_k) - \left(v_k(B_h) + v_k(f) - \min_{g \in B_h \cup \{f\}} v_k(g) \right). \tag{1}$$

The minimum is always defined because the enlarged host bundle contains f.

Theorem 3.1 (generalized parking criterion)**.** *Suppose B is* EFX_0*. For a fixed host h, the completion B^{+h} is* EFX_0 *if and only if*

$$\sigma_{k,h}(B, f) \geq 0 \qquad \textit{for every } k \neq h.$$

Consequently all n parkings are EFX_0 *if and only if every off-diagonal entry of the parking-slack matrix is nonnegative.*

Proof. Fix a host h. Consider an ordered observer–target pair after adding f. If the target is not h and the observer is not h, both bundles are unchanged from B, so the EFX_0 inequalities follow from the hypothesis. If the observer is h and the target is some $j \neq h$, the observer's own value has weakly increased while the target is unchanged, so these inequalities also remain valid.

It remains to consider observer $k \neq h$ and target h. The observer's own bundle is B_k, while the target is $B_h \cup \{f\}$. Among all possible removed goods, the largest remaining target value is obtained by removing a k-minimum good. Thus the complete family of EFX_0 inequalities for this pair is equivalent to

$$v_k(B_k) \geq v_k(B_h) + v_k(f) - \min_{g \in B_h \cup \{f\}} v_k(g),$$

which is exactly $\sigma_{k,h}(B, f) \geq 0$. This proves both necessity and sufficiency. □

When B_h is nonempty, the condition can equivalently be written

$$v_k(B_k) \geq v_k(B_h) + \max\left\{0,\, v_k(f) - \min_{g \in B_h} v_k(g)\right\}.$$

This form makes clear that a floater need not be a minimum-valued good: the base may instead provide enough slack to absorb its excess value.

Corollary 3.2 (universal-minimum special case)**.** *Suppose B is envy-free and $v_k(f) \leq v_k(g)$ for every agent k and every $g \in M \setminus \{f\}$. Then all n parkings of f are* EFX_0.

Proof. Envy-freeness implies EFX_0 under nonnegative valuations, so B meets the hypothesis of Theorem 3.1. For nonempty B_h, universal minimality makes the parenthesized target remainder in (1) equal to $v_k(B_h)$; for empty B_h it equals 0. Envy-freeness gives the required inequality in either case, and Theorem 3.1 applies. □

Neoh and Teh use the zero-valued-good special case of Corollary 3.2 in their counting analysis [2]. The corollary is also related to, but is not being attributed as a lemma of, the envy-graph and charity techniques in [6, 7, 8]. Theorem 3.1 is the direct exact criterion needed here.

Application to the witnesses. For each matrix in Section 2, the artifact identifies a good f and an EFX_0 base B such that every EFX_0 allocation of that instance is one of the n parkings B^{+h}. It records every integer $\sigma_{k,h}(B, f)$ and checks it directly from the matrix. Only the $(6, 9)$ floater is a universal minimum; no such claim is needed. Theorem 3.1 proves that the n listed parkings are EFX_0, while the exhaustive verifiers establish that there are no additional allocations. For completeness, the certified bases below are written in agent order with one-based goods:

$$\begin{array}{ll}
(4,7) & (\{1,5,6\},\{3\},\{7\},\{2\}),\\
(5,8) & (\{4\},\{3\},\{1\},\{5\},\{2,6,8\}),\\
(6,9) & (\{1,2,3\},\{4\},\{5\},\{6\},\{7\},\{8\}),\\
(4,8) & (\{3\},\{1,2\},\{6,7,8\},\{5\}),\\
(4,9) & (\{4,6,7,8,9\},\{2\},\{3\},\{5\}),\\
(4,10) & (\{3\},\{5\},\{2\},\{1,4,7,8,9,10\}).
\end{array}$$

4 Bounded counting by protected slots

We next consider the finite domain

$$\mathcal{V}_K = \{1, \ldots, K\}^{n \times m}.$$

Let $\mathcal{A} = [n]^m$ be the set of labeled allocations, represented by their good-to-agent assignment vectors, and let $E(V, A)$ denote that A is EFX_0 under valuation matrix V.

cell	floater	base shape	min. EF	min. parking	universal min.	count
$(4, 7)$	4	3111	1	1	no	4
$(5, 8)$	7	31111	3	3	no	5
$(6, 9)$	9	311111	1	1	yes	6
$(4, 8)$	4	3211	1	1	no	4
$(4, 9)$	1	5111	1	1	no	4
$(4, 10)$	6	6111	1	1	no	4

Table 2: Parking certificates for the six displayed matrices. Goods are one-based. The artifact records the full base bundles and observer–host slack matrices; "min. EF" is the minimum ordinary envy-free base margin.

4.1 The master formula

Fix an integer target lower bound $2 \le c \le |\mathcal{A}|$ and a finite pool $P \subseteq \mathcal{A}$. Introduce $c-1$ slot allocations $S_1, \dots, S_{c-1} \in \mathcal{A}$. The protected-slot master is

$$V \in \mathcal{V}_K, \qquad C(V), \qquad \bigwedge_{A \in P} \left(\neg E(V, A) \ \vee \ \bigvee_{t=1}^{c-1} (A = S_t) \right), \tag{2}$$

where $C(V)$ is an orbit-complete symmetry-breaking condition. Our CP-SAT master uses row/column DoubleLex [9]; relabeling agents or goods preserves the number of EFX_0 allocations. DoubleLex is orbit-complete: choose a lexicographically maximal row-major matrix in any row/column-permutation orbit. An out-of-order adjacent row or column could be swapped to make that matrix larger, so its rows and columns must both be lexicographically ordered.

For integer valuations, non-EFX of a fixed allocation A is encoded exactly. Writing $A_g = j$ when good g is assigned to j, we have

$$\neg E(V, A) \iff \bigvee_{\substack{g \in M,\ j = A_g \\ i \neq j}} \left[\sum_{q : A_q = i} v_{i,q} + v_{i,g} + 1 \le \sum_{q : A_q = j} v_{i,q} \right]. \tag{3}$$

The $+1$ is the integral form of strict strong envy. Equality $A = S_t$ is the conjunction of the m corresponding assignment equalities.

Lemma 4.1 (protected-slot soundness)**.** *Assume C contains at least one representative of every row/good relabeling orbit. If the master (2) is infeasible for any pool P, then every $V \in \mathcal{V}_K$ has at least c EFX_0 allocations.*

Proof. Suppose instead that some $V \in \mathcal{V}_K$ has $r < c$ EFX_0 allocations. Relabel its rows and columns to obtain a representative satisfying C; the count remains r. Assign its r EFX_0 allocations to r protected slots and fill any unused slots arbitrarily. For every $A \in P$, either A is not EFX_0, satisfying the first disjunct, or it is one of the at most r protected allocations, satisfying a slot disjunct. Hence the master is feasible, a contradiction. □

The slot assignments need not be distinct or themselves constrained to be EFX_0. Allowing arbitrary slots only relaxes the master, so it cannot create a false infeasibility conclusion.

Independent cardinality form. Because P is a set of distinct allocations, existentially eliminating the slots in (2) gives the equivalent constraint

$$\sum_{A \in P} \mathbf{1}[E(V, A)] \leq c - 1.$$

Indeed, slots can cover every surviving pooled allocation exactly when there are at most $c-1$ distinct survivors. The independent Z3 re-encoding uses this cardinality form and one-hot finite-domain value variables, rather than sharing the CP-SAT slot-model builder.

Independent CNF form. For $K = 4$, a second checker retains the literal slots and independently translates the master to CNF. It represents each value by threshold bits $y_{i,q,t} = \mathbf{1}[v_i(q) \geq t]$ for $t = 2, 3, 4$, with $y_{i,q,4} \Rightarrow y_{i,q,3} \Rightarrow y_{i,q,2}$ and $v_i(q) = 1 + \sum_{t=2}^{4} y_{i,q,t}$. Fix a disjunct of (3), let $a = |A_i|$, and let $b = |A_j \setminus \{g\}|$. Its strict-envy inequality is equivalent to

$$\sum_{\substack{q \in A_j \setminus \{g\} \\ t=2,3,4}} y_{i,q,t} + \sum_{\substack{q \in A_i \\ t=2,3,4}} (1 - y_{i,q,t}) \;\geq\; 4a - b + 1. \tag{4}$$

Indeed, the left side is $v_i(A_j \setminus \{g\}) - b + 4a - v_i(A_i)$. The checker guards PySAT's satisfiability-preserving sequential-counter encoding [14, 15] of (4) by the corresponding violation literal, gives every good in each slot a one-hot owner, and encodes the first strict coordinate of every DoubleLex comparison explicitly. Its DIMACS formula therefore shares the terminal pool and mathematical specification with the CP-SAT and Z3 checks, but not either model builder.

4.2 CEGAR loop and bounded result

Starting from a pool, CP-SAT solves (2). If the master is feasible, an exact slave enumerates the candidate valuation's EFX_0 allocations. A candidate with fewer than c is a counterexample. Otherwise the slave adds allocations to the pool. Since a feasible master permits at most $c - 1$ pooled allocations to remain EFX_0, returning at least c candidate allocations necessarily supplies a new pool member. Thus the pool strictly grows inside the finite set $\mathcal{A}$, so only finitely many such refinements are possible. If the master becomes infeasible, Lemma 4.1 yields the bounded lower bound.

Computer-verified result 4.2 (bounded $(4, 7)$ count)**.** A CP-SAT [12] execution of the protected-slot master with $n = 4$, $m = 7$, $K = 4$, and $c = 4$ reports infeasible. A fresh reconstruction terminates with a complete pool of 1,240 distinct allocations. The independently encoded literal-slot CNF for that pool is also UNSAT under CaDiCaL [16]. CaDiCaL's archived binary DRAT proof is accepted by the independent `drat-trim` checker. Conditional on the reported CNF translation and proof checker, every positive integer $(4, 7)$ instance with entries in $\{1, \ldots, 4\}$ has at least four EFX_0 allocations.

The implementation creates three slots. For each pooled allocation it adds the exact disjunction (3) or equality with one of those slots. It uses no valuation-type restriction. A separate fixed-instance test checks the off-by-one boundary: protecting three distinct EFX_0 allocations is feasible, whereas pooling all four allocations of the fixed count-4 witness is infeasible.

4.3 Reproducibility and trust boundary

The accompanying supplementary artifact freezes the claim-bearing inputs and source, with pinned external solver and proof-checker revisions. Its entry points are `repro/counting_witnesses`, `artifacts/generalized_parking`, and `repro/countcegar_k4`. It includes:

- the six valuation matrices, expected counts, two pure-Python exact verifiers, a vectorized exact counter, and a definition-faithful regression;
- the generalized-parking bases, floaters, complete slack matrices, and a standard-library certificate checker;
- an audit of the historical checkpoint, an atomic reconstruction program, the complete 1,240-allocation terminal pool, run metadata, logs, and a SHA-256 manifest; and
- an independently written Z3 [13] master checker and SMT-LIB2 exporter for any reconstructed terminal pool, together with the threshold-CNF checker, differential tests, frozen DIMACS input, compressed binary DRAT proof, pinned tool revisions, and proof-verification wrapper described above.

The archival deterministic release workflow under `supplement` binds the original source and PDF to those files. This dated specialist-review copy is versioned in the repository with its dated replay logs. The historical 5 September review manifest, source inventory and editorial diff are retained under `docs/reviews/2026-09-05`; that manifest describes the original standalone review package, not subsequent repository edits. Exploratory results elsewhere in the repository are not claims of this paper.

The historical result reports infeasibility at pool size 1,295, but its periodic checkpoint contains only 1,250 entries, 1,134 distinct. The last 45 entries cannot be recovered from the historical log. The reconstruction starts from the distinct checkpoint entries and writes every subsequent pool atomically. Any terminal pool it produces therefore determines an alternative complete terminal master for the same finite claim; it does not reconstruct the missing historical allocations.

The fresh run begins with the 1,134 distinct checkpoint allocations, makes seven feasible master calls that add 106 new allocations, and returns infeasible on its eighth call. It uses CP-SAT 9.15 with six workers and random seed 1 and takes 1,060.725 seconds in the recorded environment. The complete pool is written atomically; its SHA-256 begins `af85029ca3ec`, with the full digest recorded in the artifact manifest. A subsequent one-call replay of the frozen pool again returns infeasible and exports the exact CP-SAT model and response; the artifact keeps the pool-generation and export-replay records separately. The independent CNF has 282,029 variables and 600,624 clauses. CaDiCaL 1.9.5, called through PySAT, returns UNSAT in 68.048 wall-clock seconds in the recorded environment. The archived DIMACS file has SHA-256 prefix `ccc2aa45a571`; a cold replay that loads this frozen DIMACS without rebuilding the master reproduces UNSAT and the same solver counters. A direct CaDiCaL 1.9.5 invocation writes a 134,426,255-byte binary DRAT proof. Official `drat-trim` verifies the archived proof in 90.182 seconds using 28,212,699 resolution steps; its core uses no RAT lemmas. The proof is stored under `repro/countcegar_k4/certificates` in a 38,753,120-byte `xz` archive whose compressed and uncompressed SHA-256 digests are recorded in the manifest.

The formal content of Lemma 4.1 and equations (2)–(3) can be checked independently of either solver. The universal bounded conclusion additionally trusts that the reported masters were built as specified. Agreement between the independently built CP-SAT and CNF masters materially reduces the encoding risk, while the checked DRAT trace removes reliance on CaDiCaL's raw

UNSAT verdict. The remaining computational trust boundary is the auditable CNF translation and a correct proof checker. Akrami et al. [11] additionally formalize their theoretical SAT foundation in Lean.

5 Scope and limitations

Upper bounds, not tightness. The six explicit matrices prove upper bounds on the minimum over additive instances. Calling them "tight" would require a matching lower bound for arbitrary additive valuations. That lower bound is precisely the content predicted by Conjecture 1.2 and is not proved here. The original general-monotone conjecture is already false [11].

Bounded means bounded. Result 4.2 concerns positive integer entries in $\{1, \ldots, 4\}$ only. No theorem bounds the magnitude of a minimum-count instance, and the computation says nothing about larger or zero entries.

What is and is not formal. The generalized parking theorem and protected-slot soundness lemma are ordinary mathematical proofs. The fixed-instance counts are exhaustive computations that are inexpensive to repeat with independent code. The bounded result is a substantially larger solver computation. The fresh terminal pool is included and makes its exact input replayable; the historical terminal input remains unrecoverable. A proof-producing solver certificate is included and independently checked; no proof-assistant formalization is claimed.

Search provenance. Simulated annealing and constraint-guided local search were used to discover the matrices, in the spirit of recent computational EFX search [10]. Discovery heuristics are not used in their verification and are outside the paper's theorem claims.

6 Conclusion

The six count-n instances show that very small EFX allocation sets already occur at several cells beyond $m = n + 2$. Their common, rigorously stated mechanism is not universal-minimum parking but the observer-specific slack criterion of Theorem 3.1. The protected-slot construction gives a general way to turn an existence CEGAR master into a bounded counting master, and the $(4, 7)$, $K = 4$ computation supplies its first test case here.

The main open mathematical question addressed here is the additive specialization: prove or refute a lower bound of n for arbitrary additive valuations. The original general-monotone question has been answered negatively [11]. The present results provide explicit additive upper-bound targets, a local completion lemma, and a replayable finite verification without claiming that they settle the additive question.

Acknowledgments

The computational campaign used independently written enumeration, constraint, and structure-checking programs, together with a multi-agent assistant workflow. All mathematical and computational claims included here are scoped to the checks described in the paper and artifact.

References

[1] W. Suksompong. On the number of almost envy-free allocations. *Discrete Applied Mathematics*, 284:606–610, 2020.

[2] T. Y. Neoh and N. Teh. Understanding EFX allocations: counting and variants. In *Proceedings of AAAI*, pages 14036–14044, 2025.

[3] R. Mahara. Extension of additive valuations to general valuations on the existence of EFX. arXiv:2107.09901, 2021.

[4] B. R. Chaudhury, J. Garg, and K. Mehlhorn. EFX exists for three agents. *Journal of the ACM*, 71(1), Article 4, 2024; arXiv:2002.05119.

[5] E. Alkassar, M. Fouz, and K. Mehlhorn. Complete EFX allocations exist for four additive agents and up to nine goods. arXiv:2608.08590, 2026.

[6] R. J. Lipton, E. Markakis, E. Mossel, and A. Saberi. On approximately fair allocations of indivisible goods. In *Proceedings of EC*, 2004.

[7] B. R. Chaudhury, T. Kavitha, K. Mehlhorn, and A. Sgouritsa. A little charity guarantees almost envy-freeness. *SIAM Journal on Computing*, 50(4):1336–1358, 2021.

[8] B. Berger, A. Cohen, M. Feldman, and A. Fiat. (Almost full) EFX exists for four agents (and beyond). In *Proceedings of AAAI*, 2022.

[9] P. Flener, A. M. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, J. Pearson, and T. Walsh. Breaking row and column symmetries in matrix models. In *Proceedings of CP*, 2002.

[10] S. Brânzei. Computing envy-free up to any good (EFX) allocations via local search. arXiv:2510.05429, 2025.

[11] H. Akrami, A. Mayorov, K. Mehlhorn, S. Srinivas, and C. Weidenbach. A counterexample to EFX: $n \geq 3$ agents, $m \geq n + 5$ items, submodular valuations via SAT-solving. arXiv:2604.18216, 2026.

[12] L. Perron and F. Didier. CP-SAT, version 9.15. Google, `https://developers.google.com/optimization/cp/cp_solver/`, 2026.

[13] L. de Moura and N. Bjørner. Z3: an efficient SMT solver. In *Proceedings of TACAS*, 2008.

[14] A. Ignatiev, A. Morgado, and J. Marques-Silva. PySAT: a Python toolkit for prototyping with SAT oracles. In *Proceedings of SAT*, pages 428–437, 2018.

[15] C. Sinz. Towards an optimal CNF encoding of Boolean cardinality constraints. In *Proceedings of CP*, pages 827–831, 2005.

[16] A. Biere, T. Faller, K. Fazekas, M. Fleury, N. Froleyks, and F. Pollitt. CaDiCaL 2.0. In *Proceedings of CAV*, pages 133–152, 2024.

[17] N. Wetzler, M. J. H. Heule, and W. A. Hunt. DRAT-trim: efficient checking and trimming using expressive clausal proofs. In *Proceedings of SAT*, pages 422–429, 2014.